RELIABILITY · START ORDER · 1/10

# Timing is part of the contract.

A six-service Minecraft network showed me why start order decides if a system works at all.

10 slides · swipe

---

Anton Petnitsky · Python / agents / backend

# Six services, one login, no promises.

Velocity → Limbo → Lobby → Survival. Postgres underneath for LuckPerms. Four plugins depending on it. One Telegram bot watching it all.

**'Random' failures are undocumented dependencies.**

# The 'random' failures, deconstructed.

01 A backend booted before Postgres was ready

02 LuckPerms could not init its SQL pool — no permissions

03 Velocity lost the backend — players bounced

04 A stale session.lock blocked the world — silent dead boot

05 All transient, all fixed by order — never by magic

Every 'intermittent' was a race I had not modeled.

Anton Petnitsky · Python / agents / backend

# How the network actually boots now.

01 Postgres first — watchdog refuses before pg_isready



02 Limbo, then Velocity, then backends — ping-verified



03 Dead JVMs killed by port, not by screen name



04 Boot order recorded — replayable when it breaks again

Hope is not a boot strategy.

Anton Petnitsky · Python / agents / backend

RELIABILITY · FOR YOU · 5/10

# Your service likely has one of these.

01 A DB check that runs after sockets open

02 Health endpoints that return 200 before warming up

03 A 'random' failure with a known pattern someone cargo-culted

04 A watchdog that restarts the wrong process on the wrong name

**Pick one tonight. The incidents go down quietly.**

Anton Petnitsky · Python / agents / backend

# Two wiring diagrams.

WISHFUL

**Start order folklore**

just run start.sh
hope everyone
wakes up in order

ACTUAL

**Start order engineered**

pg_isready gate
each hop pings
failure is loud

**One diagram is a rumor. The other is a boot sequence.**

Anton Petnitsky · Python / agents / backend

RELIABILITY · LOCKS · 7/10

# The deadlock nobody watches for.

Minecraft drops a session.lock when it boots. Kill the process hard — the stale lock stays. Next boot hangs on 'already locked', forever, silently. The world is fine; the boot is cursed.

**Always clear stale locks or verify they are absent.**

Anton Petnitsky · Python / agents / backend

# The deployment law.

**Restarting a dead system without understanding boot order is rehearsing for the next incident.**

Recovery is a procedure. Write it down before 3 a.m. finds it.

**Anton Petnitsky · Python / agents / backend**

# Audit your deployment in one pass.

List every service it starts. Draw the dependency arrows. Add explicit waits where an arrow is missing. Delete every health check that lies. Then boot from cold — and watch.

**Cold boots don't lie.**
**They only reveal.**

Anton Petnitsky · Python / agents / backend

RELIABILITY · KEEP THIS · 10/10

# Save this for your next deploy review.

The best outage is the one you never see because the boot order was right. Make that boring. Boring is the whole game.

Save for later review

Anton Petnitsky · Python / agents / backend